

UNIwersytet Zielonogórski

Wydział Informatyki, Elektrotechniki i Automatyki

Praca dyplomowa

Kierunek: Informatyka

SPERSONALIZOWANA INSTALACJA SMART HOME

Mateusz Banaszkiewicz

Promotor:

prof. dr hab. inż. Larysa Titarenko

Konsultant:

prof. dr hab. inż. Alexander Barkalov

Pracę akceptuję:

.....

(data i podpis promotora)

Zielona Góra, Styczeń 2024

Streszczenie

Tematem pracy jest tworzenie instalacji Smart Home w sposób czytelny dla użytkownika, pozwalający mu na łatwe zrozumienie jego działania i rozbudowę na własne potrzeby. W materiale poruszane są kwestie innych dostępnych rozwiązań i ich negatywne aspekty, na tle których przedstawiana jest struktura projektu, stawiająca na modularność i prostotę.

Słowa kluczowe: Smart Home, wydarzenia, zarządzanie urządzeniami, rozbudowa.

Spis treści

1. Wstęp	1
1.1. Wprowadzenie	1
1.2. Cel i zakres pracy	3
1.3. Struktura pracy	3
2. Teoria	5
2.1. Przegląd gotowych rozwiązań	5
2.1.1. Pojedyncze urządzenia	5
2.1.2. Asystent wirtualny	6
2.1.3. Gotowe systemy	6
2.1.4. Projekty open source	6
2.2. Wybór technologii	7
2.2.1. Główny komputer serwera	7
2.2.2. Język serwera	8
2.2.3. Zarządzanie danymi	9
2.2.4. Aplikacja Android	10
3. Użytkowanie projektu	12
3.1. Instalacja	12
3.2. Łączenie się aplikacji z serwerem	13
3.3. Zarządzanie systemem	13
3.4. Rozbudowa dla własnych potrzeb	13
4. Szczegóły projektu	15
4.1. Fizyczna konstrukcja	15
4.1.1. Spis urządzeń	15
4.1.2. Schemat połączeń	15

4.2. Main	16
4.3. Output	17
4.3.1. Opis klasy	17
4.3.2. Led.py	18
4.3.3. MotorStepper.py	19
4.3.4. ArduinoSubBoard.py	22
4.4. Paski Ledowe	22
4.4.1. Wybór technologii	22
4.4.2. Translator napięć i podpięcie bezpośrednie	22
4.4.2.1. Translator napięć	22
4.4.2.2. Sterowanie paskami	23
4.4.2.3. Pozytywne aspekty rozwiązania	24
4.4.2.4. Negatywne aspekty rozwiązania	24
4.4.3. Sterownik Arduino	24
4.4.3.1. Połączenie I2C	25
4.4.3.2. Kod Arduino	25
4.4.3.3. Pozytywne aspekty rozwiązania	27
4.4.3.4. Negatywne aspekty rozwiązania	27
4.5. Input	28
4.5.1. Opis klasy	28
4.5.2. SensorMotion.py	29
4.6. Summon	29
4.6.1. Opis klasy	29
4.7. Trigger	30
4.7.1. Opis klasy	30
4.7.2. TriggerTime.py	31
4.7.3. TriggerSignal	31
4.8. Event	31
4.8.1. Opis klasy	31
4.9. Calendar	33
4.9.1. Opis klasy	33
4.9.2. Kolejka Eventów	33

4.9.3. Spis Triggerów	33
4.9.4. Config	34
4.9.5. MainSummons	34
4.10. Listy	34
4.10.1. Opis klasy	34
4.10.2. Importy i atrybuty	35
4.10.3. Zapis i odczyt pilku	35
4.10.4. Znajdowanie obiektów z listy	36
4.10.5. Zarządzanie obiektami	36
4.11. ConnectionHandler	37
4.11.1. Opis klasy	37
4.11.2. Obsługa połączenia klienta	37
4.11.3. Komendy	38
4.12. Aplikacja Android	39
4.12.1. MainActivity: Ekran główny	39
4.12.2. SettingsActivity: Ustawienia	40
4.12.3. Listy pozycji	40
4.12.4. AddNewActivity: Menu zarządzania	41
4.12.5. SocketManager	41
5. TODO Dyskusja rezultatów i wnioski końcowe	43
5.1. Porównanie rezultatów z oczekiwaniami	43
5.2. Wnioski	44

Spis rysunków

2.1. Podgląd pliku JSON ze spisem Outputów	10
4.1. Diagram układu.	16
4.2. Schemat dwubębnowego koła pasowego [1].	21
4.3. Struktura maszyny stanów.	26
4.4. Wygląd ekranów aplikacji.	42

Rozdział 1

Wstęp

1.1. Wprowadzenie

Dwudziesty pierwszy wiek owocuje ogromnym rozwojem technologii, sprawiającym iż przeciętny obywatel korzysta z coraz to większej ilości urządzeń elektrycznych, często z rozbudowanymi funkcjami umożliwiającymi wykonywanie przeróżnych działań. Oprócz posiadanych przez ponad 85% ludności świata smartfonów [2], wiele osób zaopatrza się w inne elektroniczne akcesoria, mające na celu ułatwić codzienne czynności. Dość popularną stała się tak zwana instalacja Smart Home.

Smart Home - *Dom wyposażony w przyrządy, które mogą być sterowane na odległość przy pomocy urządzenia podłączonego do internetu.* [3]

Według statystyk na rok 2023 liczba mieszkań korzystających z urządzeń Smart Home wynosi ponad 300 milionów. Szacuje się, że do roku 2028 wartość ta urośnie do prawie 800 milionów [4]. Z tego wynika że przeciętni obywatele coraz to bardziej zainteresowani są sprytną domową elektroniką, która asystowałaby im w codziennym życiu.

Największą popularnością cieszą się wirtualni asystenci, czyli sprzęty z załączonym AI, takimi jak Amazon Alexa, Google Assistant czy Siri [5], mające odpowiadać na pytania i służyć pomocą w prostych czynnościach jak obsłudze urządzeń wcho-

dzących w skład naszego Smart Home, czyli na przykład telewizora, klimatyzacji, odkurzacza, oświetlenia itp. [6]

Większość tego typu akcesoriów nie posiada jednak integracji z takimi asystentami, i jedyny sposób na zarządzanie nimi to korzystanie z aplikacji mobilnych stworzonych specjalnie pod nie. W efekcie osoba posiadająca sporą kolekcję takich urządzeń kończy z multum przeróżnych oprogramowań zaśmiecających telefon, często wymagających osobnych kont, haseł i różnego rodzaju pozwoleń. To samo w sobie potrafi odrzucić niejednego potencjalnie zainteresowanego własną instalacją, nie wspominając o frustracji, gdy porządana funkcja nie istnieje bądź działa nie do końca zgodnie z oczekiwaniami. Dodatkowo niektórzy mogą wykazywać niechęć do wirtualnych asystentów, ze względu na potencjalnie intruzywny charakter takich urządzeń, gromadzących dane czy nawet zapisy głosowych wypowiedzi użytkownika [7].

Z tego właśnie powodu narodził się przedstawiony w tym materiale projekt. Miał on być solidną, a zarazem prostą bazą, która będzie umożliwiała użytkownikowi dodanie własnych modułów oraz zarządzanie nimi w jednym miejscu, integrując wszystkie elementy w jeden system. Zaletami takiego rozwiązania są:

- **Pełna prywatność** - Użytkownik ma pewność o bezpieczeństwie swoich danych, gdyż cały system jest prosty i zrozumiały oraz możliwy do inspekcji i modyfikacji na własne potrzeby.
- **Wymagana funkcjonalność** - Zarządzając modułami użytkownik ma możliwość nie tylko dostosowania ich funkcji pod własne potrzeby i wizje, ale też utworzyć nowe, w pełni realizujące jego oczekiwania.
- **Prostota i dostępność** - Jeden główny system zajmujący się wszystkimi modułami które użytkownik do niego podłączył oraz jedna aplikacja nie wymagająca tworzenia konta ani niepotrzebnych pozwoleń to minimalistyczne podejście, które likwiduje niepotrzebne komplikacje i gwarantuje przyjemność z korzystania z systemu.

1.2. Cel i zakres pracy

Projekt powstał w celu umożliwienia użytkownikowi stworzenia własnego systemu Smart Home z gotowych lub jego własnych elementów, które mogą być dostosowane i skonfigurowane do indywidualnych potrzeb. Aby udowodnić użyteczność oraz funkcjonalność tego podejścia, zdecydowano się na zbudowanie wykorzystującej go instalacji, mającej za zadanie posiadać następujące funkcje:

- Zawierać podświetlenie adresowalnych pasków LED, czujniki ruchu, silniki krokowe oraz inne elementy.
- Umożliwić użytkownikowi kontrolę nad kolorami i jasnością poszczególnych pasków oraz zapewnić mu sposobność dodawania własnych animacji i sekwencji animacji.
- Zezwolić na kontrolę rolet okiennych za pomocą silników krokowych.
- Posiadać aplikację na telefon z systemem Android. Użytkownik za jej pomocą ma możliwość sterowania systemem oraz zarządzać zaplanowanymi wydarzeniami, które wykonują się o określonym czasie lub po spełnionych przez czujniki wymaganiach.

1.3. Struktura pracy

Ten materiał został podzielony na rozdziały, każdy traktujący o innym aspekcie projektu:

- **2 Teoria** - Opisanie przydatności i unikalności projektu oraz objaśnienie decyzji związanych z zastosowanymi rozwiązaniami.
- **3 Użytkowanie** - Instrukcje wyjaśniające w jaki sposób należy korzystać z projektu.
- **4 Szczegóły** - Wnikliwa analiza wszystkich elementów projektu oraz zagłębienie się w strukturę i mechanizmy każdego z nich.

- **5 Wnioski** - Omówienie najważniejszych informacji o projekcie, wiedzy zdobytej z przeprowadzonej analizy oraz udowodnienie spełnienia wymagań postawionych w punkcie 1.2

Rozdział 2

Teoria

2.1. Przegląd gotowych rozwiązań

Aby w pełni zrozumieć zapotrzebowanie na przedstawiony w tym materiale projekt warto jest przyjrzeć się dostępnym obecnie pozycjom oferującym podobne usługi. Dzielą się one na kilka kategorii:

2.1.1. Pojedyncze urządzenia

Tak jak wspomniano w punkcie 1.1, wiele osób zaczynających swoje obcowanie ze Smart Home’em rozpoczyna od kolekcji akcesorii różnych producentów i obsługuje je indywidualnie. Przykładami urządzeń najchętniej kupowanych są: telewizory, kamery monitoringu, odkurzacze, przełączniki światła, systemy podlewania ogrodowego, oświetlenie czy głośniki [8].

Dość dużym minusem przy tego typu postępowaniu jest proporcjonalnie zwiększająca się trudność kontroli wraz ze wzrostem liczby posiadanych przedmiotów, wymagających często osobnych aplikacji. Dodatkowo użytkownicy kierujący się tym podejściem bardzo często zainteresowani są pojedynczymi, konkretnymi urządzeniami, za to cenią sobie bardzo dużą swobodę w korzystaniu z nich, co czasami nie jest zapewnione przez dostarczone oprogramowanie i jest trudne do obejścia.

2.1.2. Asystent wirtualny

Również napomniane wcześniej w punkcie 1.1 sprzęty, obsługujące sztuczną inteligencję znaną jako wirtualny asystent, potrafią rozbudować poprzednią metodę. Dodają one bowiem główny element, pozwalający na połączenie wszystkich pozostałych maszyn w jeden system. Główną zaletą tego przedsięwzięcia jest otrzymanie centralnej jednostki, dzięki której można sterować całym układem z jednego miejsca.

Niestety jednak zazwyczaj tacy asystenci wspierają jedynie akcesoria tego samego producenta, co znacznie zmniejsza możliwości użytkownika, bądź zmusza go do wymiany już posiadanych instrumentów na rzecz kompatybilnych. Lecz jeszcze większą wadą jest niechęć do tego rozwiązania związana z udostępnianiem dużej ilości swoich danych tym narzędziom [7].

2.1.3. Gotowe systemy

Bardziej profesjonalnym podejściem jest zakup całego zestawu technologii od konkretnego producenta, który cały układ projektuje specjalnie pod umieszczone w nim elementy. Często takie rozwiązania przychodzą ze znacznie rozbudowaną jednostką centralną, pozwalającą na dużo większą kontrolę i dostęp do danych zbieranych przez system. Przykładami marek takich produktów są Ezlo, Notion albo Loxone.

Jest to świetna technologia dla osób chcących mieć sprawdzony i rozbudowany Smart Home, niebędących przy tym zainteresowanymi jeszcze większą personalizacją od tej zapewnionej przez wybraną przez nich firmę.

2.1.4. Projekty open source

Istnieje dość liczne grono osób, które cenią sobie przede wszystkim pełnię personalizacji oraz kontroli. Żadna z wyżej wymienionych metod nie oferuje możliwości inspekcji ani zmiany oprogramowania urządzeń czy systemów. Z tego powodu entuzjaści z całego świata tworzą własne projekty **open source**.

Open source - *oprogramowanie, dla którego oryginalny kod źródłowy jest udo-*

stępniany bezpłatnie i może być rozpowszechniany i modyfikowany bez uiszczania opłat. Oprogramowanie typu open source (OSS) to dowolne oprogramowanie komputerowe rozpowszechniane z kodem źródłowym dostępnym do modyfikacji.[9]

Ze względu na działanie takich projektów, wiele z nich gromadzi sporą liczbę osób regularnie pracujących nad ich funkcjonalnością, dodając lub udoskonalając wsparcie dla przeróżnych urządzeń, metody mogące być wykorzystane przez użytkownika bądź systemy zabezpieczeń chroniących dane przed atakami hakerskimi. Tego typu struktury są świetne dla osób chcących zbudować swój własny system, jednocześnie zaoszczędzając, gdyż jedyne koszty do poniesienia to zakup odpowiednich części i elementów.

Jednak to podejście może odstraszać odbiorców którzy nigdy wcześniej nie mieli styczności z tworzeniem tego typu układów ani nie zajmowali się w dużym stopniu bądź w ogóle programowaniem. Duże, rozbudowane projekty, często utrzymywane przez wiele osób z różnymi podejściami rzadko kiedy są czytelne i proste do zrozumienia dla kogoś chcącego zacząć swoją przygodę z urządzeniami Smart Home bądź systemami wbudowanymi generalnie. Właśnie dla takich osób powstał projekt przedstawiony w tym materiale.

2.2. Wybór technologii

Zgodnie z ideą projektu traktującą o dostępności, przejrzystości i ułatwionej rozbudowie, istotnym był wybór zastosowanych rozwiązań. W tym punkcie przedstawiono oraz uzasadniono dokonane decyzje.

2.2.1. Główny komputer serwera

Cała struktura projektu opiera się na jednym, centralnym kontrolerze zwanym serwerem, zarządzającym urządzeniami peryferyjnymi oraz innymi podprocesami. Ważna jest wszechstronność tego modułu, pozwalająca na różne podejścia, oraz dość duża moc obliczeniowa, umożliwiającą tworzenie programu w sposób przejrzysty, bez potrzeby martwienia się o zasoby i stuprocentową optymalność.

Dobrym kandydatem spełniającym podane wymagania okazały się komputery **Raspberry Pi**, a model **4B 8GB** został wykorzystany w przykładowej instalacji. Urządzenia z tej serii cechują się bardzo dużą popularnością wśród początkujących i zaawansowanych, tworzących własne elektroniczne projekty, dzięki czemu praktycznie każde zagadnienie związane z użytkowaniem posiada dużą część społeczności aktywnie dyskutującą i przedstawiającą optymalne rozwiązania. Płytki te cechują się szerokim wachlarzem portów oraz pinów GPIO (złącz wejścia/wyjścia, służących do komunikacji między elementami systemu komputerowego [10]), zezwalających na fachową rozbudowę, a podzespoły wybranego modelu bez problemu spełniają wymagania projektu.

2.2.2. Język serwera

Wybór języka programowania w którym będzie pisany system jest bardzo istotny. Aby sprostać temu zadaniu, należy zastanowić się nad celem, jaki miałyby on pomóc realizować. W przypadku omawianego projektu, wysoko porządzaną cechą jest prostota i przejrzystość, umożliwiającą korzystanie i zagłębianie się w strukturę programu nawet użytkownikowi, który nie zajmował się nigdy wcześniej programowaniem.

Ze względu na wcześniej wybraną główną jednostkę, ta kwestia praktycznie została rozwiązana sama, gdyż Raspberry Pi słynie ze swojego wsparcia dla języka **Python**, który jest najpopularniejszym wyborem do pisania skryptów na tej właśnie platformie [11]. Jest on znany ze swojej prostoty, wszechstronności, oraz nieustannie znajduje się w czołówce najchętniej wybieranych języków przez początkujących programistów [12].

Dodatkowym atutem jest ogromny wybór różnego rodzaju bibliotek, pomagających osiągnąć przeróżne cele z łatwością. Jest to szczególnie pomocne przy zarządzaniu różnymi elementami Raspberry Pi, które utrzymuje wspomniane wcześniej wsparcie w tym kierunku dla tego języka. Dlatego w projekcie zdecydowano się na wykorzystanie **Pythona 3** jako podstawowy język systemu.

2.2.3. Zarządzanie danymi

Każdy system Smart Home, bądź inny projekt zarządzający swego typu automatyzacją muszą mieć dostęp do instrukcji, które pozwalają im na prawidłowe funkcjonowanie. To tylko przykład z wielu różnych sytuacji, w których to zastosowanie ma baza danych, czyli moduł zajmujący się magazynowaniem i zarządzaniem wszelkiego rodzaju informacjami, które są później wymagane przez przeróżne części systemu. W przedstawionym projekcie takimi danymi są między innymi spisy urządzeń podłączonych przez użytkownika oraz zaprogramowane przez niego wydarzenia wraz z warunkami wykonania się. Zapisywanie tych wartości jest kluczowe, gdyż pozwalają na utrzymanie stanu wszystkich porządzanych przez układ zmiennych pomiędzy na przykład restartami systemu.

Istnieje wiele gotowych rozwiązań w dziedzinie baz danych, z wiodącymi nazwami takimi jak Oracle, MySQL czy MongoDB [13]. Jednak korzystanie z nich wiąże się z instalacją i dość skomplikowaną konfiguracją dość specyficznych modułów. Oczywiście te oprogramowania są niezastąpione przy masywnych i skomplikowanych systemach, gdyż charakteryzują się znakomitą efektywnością zarówno w przechowywaniu jak i zarządzaniu danymi, lecz na potrzeby omawianego projektu są po prostu zbyt skomplikowane i trudne w obsłudze jak na funkcje które mają pełnić, szczególnie dla użytkownika chcącego rozbudować czy zinspektować taki moduł.

Z tego właśnie powodu zdecydowano się na utworzenie **własnego systemu bazy danych** (dokładniej opisanego w punkcie 4.10), który potrzebne dane zapisuje w poszczególnych plikach **JSON**. Cała struktura wymiany danych pomiędzy serwerem oraz aplikacją również oparta jest o ten właśnie format. Cechy takiego rozwiązania to przede wszystkim prostota, ponieważ zapis i odczyt pliku zostaje wykonany przez dwie nieskomplikowane funkcje, oraz przejrzystość, gdyż każdy z tych plików można dowolnie otworzyć i podejrzeć, by na bieżąco śledzić poprawność działania systemu.

Minusem tej metody jest dość łatwy dostęp do danych osób trzecich, gdyż pliki nie są w żaden sposób szyfrowane. Każdy kto uzyska dostęp do serwera, może podejrzeć zawarte w nim informacje (rysunek 2.1). Jednak biorąc pod uwagę, iż żadne poufne ani osobiste dane nie znajdują się (a przynajmniej nie powinny) w tych plikach, uznać można, że nie ma zbyt wielkiej potrzeby chronienia ich. Użytkownik czujący jednak konieczność zabezpieczenia swoich danych może dokonać odpowied-

nich modyfikacji.

```
},
{
  "id": 9,
  "name": "Roleta okienna",
  "classType": "MotorStepper",
  "classArgs": {
    "pins": [
      18,
      23,
      24,
      25
    ],
    "savedPositions": {
      "otwarta": 246,
      "zamknieta": -129
    },
    "currentSteps": 145
  },
  "mainSummon": null
},
}
```

Rysunek 2.1. Podgląd pliku JSON ze spisem Outputów

2.2.4. Aplikacja Android

Aplikacja mobilna będąca istotną częścią projektu jest zarazem całkowicie osobnym elementem. Mimo iż razem z serwerem stanowią jego całość, powstała ona na zupełnie innych zasadach, oraz w innym języku. Dlatego rozwiązania poszczególnych aspektów dają pewną wolność, pozostając równocześnie wielce istotnymi. Prawdopodobnie najważniejszą kwestią odnośnie tworzenia aplikacji na platformę Android jest wybór odpowiedniego IDE (Zintegrowanego środowiska programistycznego [14]). Do wykonania projektu wykorzystano **Android Studio**, jeden z popularniejszych programów, należącego i rozwijanego przez Google oraz JetBrains.

Wybór ten uzasadniony jest dość szerokim zakresem poradników na różne tematy tworzenia aplikacji, oraz prostym faktem, iż po zainstalowaniu tego oprogramowania wymagana ilość konfiguracji jest znikoma, dzięki czemu nowicjuszowi łatwiej jest

zacząć pracę nad projektem. Dodatkowym atutem jest zintegrowany emulator, który pozwala na nieustanne testowanie budowanej aplikacji na prawdziwym systemie Android, dzięki czemu wyłapywanie błędów i nanoszenie poprawek jest w znacznym stopniu ułatwione.

Android Studio oferuje możliwość tworzenia aplikacji w dwóch językach: Kotlinie oraz Javie. Oba języki cieszą się dużą popularnością w śród Androidowych deweloperów, jednak to **Java** jest dużo większym przedmiotem zainteresowania wszelkiego rodzaju forum programistycznych, co przekłada się na ogromny zasób poradników oraz dyskusji nad optymalnymi rozwiązaniami. Z tych że powodów to ona została użyta jako język programowania aplikacji.

Kolejną ważną kwestią jest sposób komunikowania się klienta (aplikacji) z serwerem. Projekt został zrealizowany w oparciu o **WebSockets**, które pozwalają na stałe połączenie oraz przesył danych w obie strony. Jest to eleganckie rozwiązanie, pozwalające na kontrolę ilości użytkowników oraz osiąganie różnych efektów w zależności od adresu IP klienta. Ta metoda zapewnia szerokie spektrum możliwych ścieżek jakie może obrać użytkownik chcący rozbudować projekt o dodatkowe funkcje.

Rozdział 3

Użytkowanie projektu

3.1. Instalacja

Aby móc poprawnie korzystać z projektu należy wypakować pliki na swoim urządzeniu Raspberry Pi 4 Model B. Inne wersje tych komputerów nie zostały przetestowane i nie zaleca się ich używania.

W `ConnectionHandler.py` należy ustawić port na którym nasłuchiwać będzie serwer (domyślnie jest to 8080). Zalecane jest również wejście w ustawienia swojego routera, oraz utworzenie zasady nadającej RP (Raspberry Pi) zawsze to samo stałe IP. Dzięki temu nigdy nie będziemy mieli kłopotów z połączeniem się do serwera poprzez błędny adres.

Należy upewnić się, że wszystkie biblioteki używane przez RP do obsługi języka Python 3 oraz pinów GPIO są zainstalowane na urządzeniu.

Aby wystartować serwer, wystarczy uruchomić plik `Main.py` znajdujący się w głównym folderze projektu. Jeżeli chcemy, by serwer startował samoczynnie przy włączeniu się RP, możemy wykonać następujące kroki:

- Otworzyć terminal.
- Wpisać komendę: `sudo crontab -e` i podać hasło.
- Na samym dole dokumentu dopisać:

```
@reboot python3 ścieżkaFolderuProjektu/Main.py &
```

Aby zainstalować aplikację mobilną na swoim urządzeniu Android należy uruchomić `smarthome_installer.apk`.

3.2. Łączenie się aplikacji z serwerem

Aby po włączeniu aplikacji nawiązać połączenie z serwerem, należy:

- Kliknąć przycisk **Settings**.
- W odpowiednich polach wpisać IP oraz PORT Serwera (Patrz punkt 3.1)
- Kliknąć przycisk **Connect**.

Po udanym połączeniu pojawi się odpowiednia informacja.

3.3. Zarządzanie systemem

Po nawiązaniu połączenia z serwerem można zsynchronizować dane, co pobierze informacje o wszystkich Eventach, Summonach, Outputach i Inputach (Opisanych w rozdziale 4). Wszystkie pozycje wraz z detalami wyświetlą się jako lista po kliknięciu w jeden z odpowiadających im przycisków na głównym ekranie aplikacji.

Chcąc dodać nową pozycję, bądź zmodyfikować lub usunąć istniejącą, należy kliknąć przycisk **Manage**. W nowo otwartym menu będzie można wykonać wszystkie potrzebne operacje. W zależności od wybranej klasy oraz typu operacji pojawiają się oraz odblokowują wymagane pola. O stanie wykonania operacji informują pojawiające się powiadomienia.

Dodatkowo w menu **Settings** istnieje dodatkowa opcja manualnego wysyłania komend do serwera.

3.4. Rozbudowa dla własnych potrzeb

System został zaprojektowany tak, by użytkownik w prosty sposób mógł dodać własne mechanizmy bez potrzeby zagłębiania się we wszystkie klasy oraz sposoby ich działania. Dbając o zawarte wymagane funkcje, można dodać między innymi

własne klasy Inputów i Outputów, pamiętając przy tym o zaimportowaniu nowo utworzonych modułów do odpowiednich części projektu. Praktycznie każdy element zaprezentowany w rozdziale 4 można dostosować do własnych potrzeb.

Rozdział 4

Szczegóły projektu

4.1. Fizyczna konstrukcja

4.1.1. Spis urządzeń

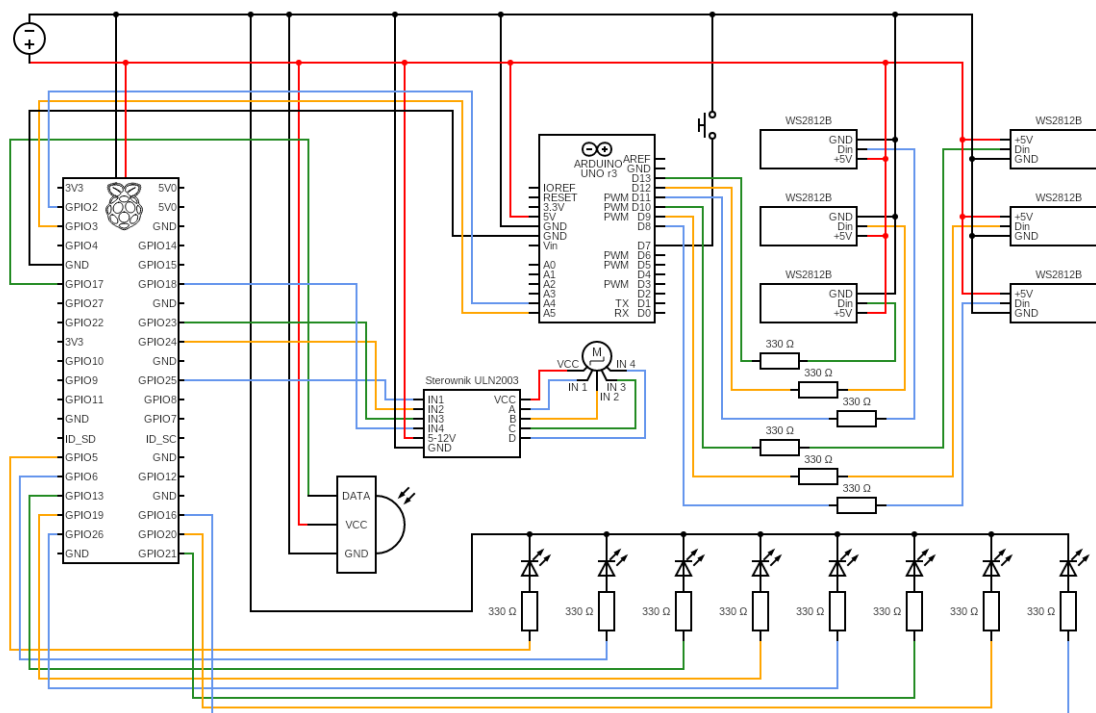
W instalacji wykonanej na potrzeby tego materiału użyto następujących modułów:

- Raspberry Pi 4 Model B 8GB Ram
- Arduino Uno
- Ledy Adresowalne WS2812B 60led/m Black PCB
- Silnik krokowy 28BYJ-48 5V ze sterownikiem ULN2003
- Rezystory 330 Ω 0,25W
- Diody LED 5mm
- Zasilacz Impulsowy LED 5V 150W 30A
- Czujnik Ruchu PIR HC-SR501

4.1.2. Schemat połączeń

Rysunek 4.1 przedstawia fizyczne podłączenie wszystkich elementów. Warto przy tym zaznaczyć, że Raspberry Pi jest zasilane przy użyciu portu USB-C. W ten

sposób można korzystać z dostarczanego przez zasilacz napięcia 5V. Inne podpięcie wiązałoby się z ryzykiem uszkodzenia pinów bądź całej płytki.



Rysunek 4.1. Diagram układu.

Bardzo ważnym aspektem projektu jest unikanie zasilania elementów przez kontroler. Im większe wymagania ma podłączone urządzenie pod względem natężenia, tym większa szansa na uszkodzenie, a gwarantowane błędne działanie. Dlatego tego typu akcesoria jak paski LED czy silniki krokowe muszą czerpać moc bezpośrednio z zasilacza. Natomiast niewymagające elementy takie jak pojedyncze diody LED można podłączać bezpośrednio do kontrolera.

Przedstawiony schemat ilustruje jedynie przykład instalacji, którą wykonano na potrzeby tego materiału. Użytkownik może dowolnie dodawać oraz odejmować moduły tak długo, jak będzie to robił poprawnie.

4.2. Main

Main jest głównym plikiem projektu. Chociaż nie zawiera wielu linijek kodu, jest nadzwyczaj istotny, gdyż uruchomienie właśnie jego włącza serwer. Jeden główny

plik jest dużo lepszym rozwiązaniem niż kilka mniejszych, które może zaoszczędzić trochę miejsca, bądź wizualnie uprościć strukturę systemu, ponieważ użytkownik nie musi się martwić ani o kolejność uruchamiania programów, ani o przypadkowe pominięcie któregoś z nich.

Plik `Main.py` posiada trzy funkcje:

- **userConsole** - Zarządza decyfrowaniem poleceń użytkownika z konsoli.
- **startup** - Uruchamia serwer.
- **shutdown** - Zatrzymuje działanie serwera.

Na samym dole klasy zobaczyć można wywołanie funkcji `startup`, które to rozpoczyna cały proces.

Main po uruchomieniu obsługuje również polecenia brane od użytkownika przez konsolę. Nie jest to narzędzie zbyt skomplikowane, ale stanowi prosty sposób na sprawdzenie poprawności działania systemu i ładowania się list.

4.3. Output

4.3.1. Opis klasy

Output jest klasą odpowiadającą za zarządzanie widocznymi zmianami którymi chce osiągnąć użytkownik. Sama klasa `Output.py` nie jest zbyt rozbudowana, ale jej struktura pozwala na wykorzystanie różnych urządzeń w ten sam sposób, bez konieczności zmieniania kodu reszty programu.

Output posiada cztery atrybuty:

- **id** - Unikalne ID, pozwalające na specyfikację o który obiekt chodzi.
- **name** - Nazwa podana przez użytkownika, dzięki której wie on co to za obiekt.
- **outputClass** - Właściwy obiekt urządzenia którym zarządza Output.
- **mainSummon** - ID Summonu, który wykonuje się w zapętleniu, do czasu gdy wydarzy się Event nakazujący inny Summon. Może być pusty, przez co nie będzie się wykonywało nic.

Oprócz nich, klasa ta ma trzy funkcje:

- **__init__** - Konstruktor, który przyjmuje nazwę, typ urządzenia wyjścia oraz główny Summon. Możliwe jest też podanie numeru id, co jest konieczne podczas wczytywania utworzonych już Outputów z pliku JSON. Natomiast gdy id nie jest podane, wartość jest brana z porównania z Configiem z Calendar.
- **action** - Główna funkcja klasy. Otrzymuje ona nazwę funkcji oraz jej argumenty, następnie wykonuje tą funkcję na urządzeniu zdefiniowanym w `outputClass`. To pozwala np.: zapalić lub zgasić diodę. Jeżeli podana funkcja nie istnieje w klasie, blok `try/catch` to wychwytuje i zwraca odpowiedni komunikat.
- **listFunctions** - Funkcja która sprawdza wszystkie możliwe do wywołania funkcje z obiektu `outputClass`. Jest przydatna, gdy mamy problem z wykorzystaniem funkcji `action`.

Ze wszystkich atrybutów klasy najistotniejszym jest bez wątpienia `outputClass`. Jest to obiekt klasy urządzenia, na przykład diody LED, która jest przypisana do tego Outputu. Poniżej opisane są już gotowe, załączone w projekcie klasy.

4.3.2. Led.py

Prosta klasa służąca do zarządzaniem pojedynczą diodą LED. Ma ona tylko jeden atrybut, którym jest `pin`. Określa on fizyczny pin GPIO płytki Raspberry Pi do którego podłączona jest dioda. Domyślnie jest to pin 12. Oprócz tego, są tu też następujące funkcje:

- **__init__** - Konstruktor, który ustawia podany pin. Oprócz tego, tworzy on globalną zmienną `pwm_led`. PWM (Pulse Width Modulation) służy do kontroli napięcia na danym pinie. Dzięki temu można uzyskać wartości pośrednie, a nie tylko 0 i 1. Warto również zaznaczyć, iż `pwm_led` nie jest atrybutem klasy celowo. Nie jest to nic potrzebnego do utworzenia klasy, w przeciwieństwie do numeru pinu. W ten sposób nie zapisuje się on także w pliku JSON, co też nie byłoby możliwe, gdyż nie jest to wartość którą można w ten format wpisać. Używając zmiennych globalnych, unika się więc błędów i niepotrzebnych danych.

- **turnOn** - Zapala diodę na pinie pełną jasnością. Można też zauważyć iż ta funkcja przyjmuje id klasy Output, ale nie wykorzystuje go. Wy tłumaczenie znajduję się w opisie funkcji slideAnimation.
- **turnOff** - Całkowicie gasi diodę.
- **slideAnimation** - Ta funkcja wykonuje animację, podczas której dioda stopniowo się rozjaśnia, a następnie ściemnia, z podaną prędkością w wartości **speed**. Można zauważyć, że w każdej iteracji pętli **for** sprawdzana jest flaga **mainSummon** z klasy Calendar. Tutaj właśnie przydaje się **outputId**. Jeżeli nie jest ono dane, nie trzeba nic robić. Natomiast jeżeli jest, oznacza to, iż ten konkretny Output, w którym jest ta dioda, ma ustalony **mainSummon**, i jest nim właśnie ta funkcja. Z założenia **mainSummon** powinien się natychmiast przełączać, jeżeli zostanie uruchomiony inny Summon na tym Outpucie i właśnie to ma na celu sprawdzanie flagi z klasy Calendar. Jeżeli ta funkcja wykryje, iż powinno się zakończyć, robi to, dając miejsce innym.
- **setBrightness** - Ustawia jasność diody na podany procent, sprawdzając przy tym, czy wartość znajduję się w zakresie 0 - 100.
- **cleanup** - Funkcja która “czyści” GPIO. Jej uruchomienie powoduje odłączenie wszystkich urządzeń na pinach, dlatego nie powinno się jej używać do momentu zamknięcia serwera. Nie jest konieczna, ale to dobra praktyka, dzięki której jeżeli użytkownik chce korzystać z GPIO w innych programach bez ponownego uruchamiania płytki, nie napotka błędów, gdyż będzie zaczynał na świeżo.

4.3.3. MotorStepper.py

Ta klasa służy do zarządzania czteropinowym silnikiem krokowym. W przykładowej instalacji użyto silnika **28BYJ-48 5V** ze sterownikiem **ULN2003**.

Silnik krokowy różni się od silnika zwykłego głównie w zasadzie działania i sposobie sterowania. Ten pierwszy porusza się krokowo, obracając się o określony kąt. Dzięki temu jest bardziej precyzyjny, utrzymuje moment obrotowy nawet przy zatrzymaniu i łatwiej go kontrolować za pomocą sygnałów impulsowych. Z kolei silnik

zwykły obraca się ciągle. Można w nim jedynie kontrolować prędkość obrotu oraz długość pracy, co jest przydatne, ale nie w precyzyjnych zadaniach.

Klasę w projekcie oparto na poradniku Gavena MacDonalda[15]. Posiada ona trzy atrybuty:

- **pins** - Czteroelementowa lista, posiadająca numery pinów do których podpięty jest silnik jako elementy w tablicy. Domyślnie są to piny 18,24,25,26.
- **savedPositions** - Słownik który trzyma informacje o konkretnych pozycjach, na które może poruszyć się silnik. Każdy element w słowniku ma klucz jako nazwę pozycji, oraz wartość w postaci liczby kroków od 0 które ma wykonać silnik by tam dojść.
- **currentSteps** - obecna pozycja silnika. Domyślnie jest to 0. Ważnym jest by zapisać tą wartość do pliku JSON, tak by silnik się nie zdesynchronizował pomiędzy restartami serwera.

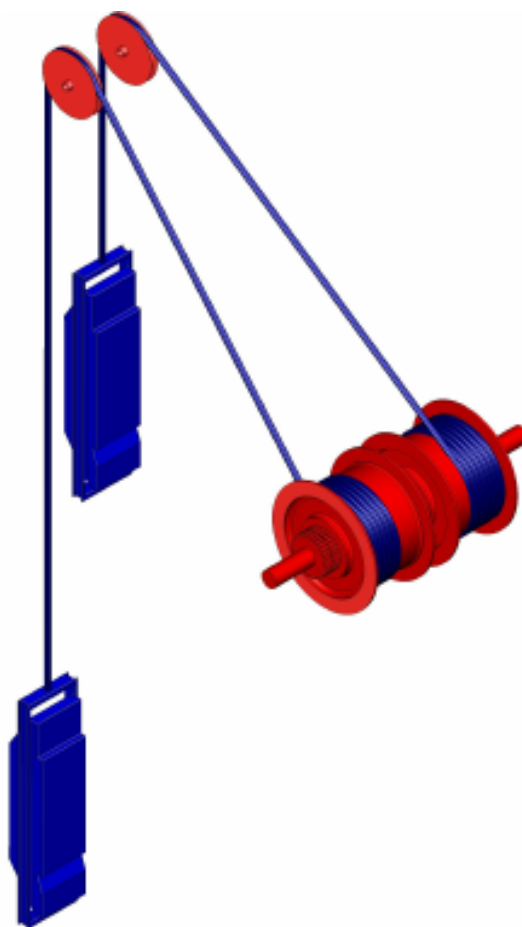
Funkcje klasy prezentują się następująco:

- **__init__** - Konstruktor, który ustawia pozycje oraz przygotowuje piny GPIO, oraz tworzy globalną zmienną **sequence**, która jest później używana do poprawnego działania silnika. Podobnie jak w przypadku Led.py, globalna zmienna jest tutaj lepszym rozwiązaniem. Można by co prawda zapisać ją w języku JSON, ale jest ona w każdej instancji klasy taka sama, więc zbędna przy zapisie.
- **move** - Porusza silnik o podaną liczbę kroków. Dla każdego kroku należy wykonać 8 “półkroków”, w których to należy ustawić odpowiednie napięcie na każdym pinie. Właśnie te wartości określa zmienna **sequence**. Jeżeli pożądanym jest obrót w przeciwnym kierunku, trzeba odwrócić kolejność półkroków. Warto też dodać iż dla każdego półkroku należy zostawić odpowiednią ilość czasu by fizyczne magnesy w silniku zdążyły się poprawnie ustawić. Dlatego program czeka jedną milisekundę za pomocą funkcji **sleep**.
- **moveTo** - Porusza silnik do danej pozycji, jeżeli znajduje się ona w słowniku **savedPositions**. Aby wyliczyć liczbę potrzebnych kroków wystarczy od zamierzonej pozycji odjąć tę obecną (jednostką są kroki).

- **cleanup** - Podobnie jak w `Led.py`, funkcja czyszcząca piny GPIO, której nie powinno się używać do momentu wyłączenia serwera.

Taki silnik jest dobrym kandydatem do automatycznego ustawiania na przykład rolet okiennych w odpowiedniej pozycji. Jako że każdy użytkownik posiada inaczej zbudowane rolety, trudnym jest przygotowanie rozwiązania które sprawdzi się w każdej konfiguracji. Jednak pewne podejścia mogą okazać się skuteczne:

- **Wydruk elementu na drukarce 3D** - Osoba mająca doświadczenie z projektami 3D mogłaby bez większego trudu zaprojektować część która pozwoliłaby na połączenie silnika z roletą.
- **Zastosowanie dwubębnowego koła pasowego** - Rysunek 4.2 przedstawia schemat, na którym dwie liny kontrolują poruszanie się ciężarów w zależności od kierunku obrotu. Ten mechanizm może zostać wykorzystany przy ciągnięciu silnika za sznurki niektórych rodzajów rolet, tak by działał on w obie strony.



Rysunek 4.2. Schemat dwubębnowego koła pasowego [1].

4.3.4. ArduinoSubBoard.py

Ta klasa odpowiada za sterowanie sześcioma paskami adresowalnymi LED, a właściwie za komunikację z konkretnie zajmującą się tym płytką Arduino Uno. Temat ten jest na tyle zawiliły, iż szczegóły zostały przedstawione w osobnym punkcie 4.4.

4.4. Paski Ledowe

4.4.1. Wybór technologii

Pasek adresowalnych ledów WS2812B posiada trzy złącza:

- **Vin** - Zasilanie przyjmujące napięcie 5V.
- **Ground** - Złącze masy.
- **Data** - Kanał transferu danych. Tym kablem przesyłane są informacje o sposobie zapalania się poszczególnych ledów. Wymagane napięcie to 5V.

Istnieje kilka różnych sposobów na obsługiwanie tych ledów za pomocą Raspberry Pi. Jednakże podczas rozważania ich, trzeba pamiętać o kluczowym aspekcie: komputer ten pracuje na napięciu 3.3V i taki też jest sygnał wysyłany przez porty GPIO. Z tego powodu niemożliwym jest osiągnięcie pozytywnych rezultatów bez użycia środków mogących zapobiec tej różnicy. W punktach 4.4.2; 4.4.3 przedstawiono dwa rozwiązania zaimplementowane w projekcie.

Bez względu na zastosowaną metodę, należy pamiętać o umieszczeniu odpowiednich rezystorów przed przesłaniem sygnału na pin **Data** paska, gdyż w przeciwnym wypadku ledy mogą ulec uszkodzeniu.

4.4.2. Translator napięć i podpięcie bezpośrednie

4.4.2.1. Translator napięć

Eliminując różnicę w napięciach można "bezpośrednio" podłączyć paski do pinów sterujących Raspberry Pi: utrzymywanie procesu sterowania każdym paskiem przez ten właśnie kontroler. Rozwiązaniem może być zastosowanie translatora napięć, czyli

urządzenia podwyższającego bądź zmniejszającego napięcie sygnału. Niektóre z tych produktów umożliwiają nawet dwustronną translację. Ze względu na używany model zasada działania może się różnić, ale w większości przypadków należy dostarczyć napięcia o obu wartościach jako swego typu punkty odniesienia, dzięki którym kontroler wie, jak ma zmodyfikować sygnał. Po podłączeniu również masy, można korzystać z portów wejścia i wyjścia urządzenia by przysyłać przez nie sygnał, który zostanie przez nie przetłumaczony.

4.4.2.2. Sterowanie paskami

Jeżeli dopilnowano by sygnał wydostający się z GPIO Raspberry Pi dochodził do pasków w wymaganym napięciu 5V, należy nadać mu odpowiedni kształt. Mikrokontrolery znajdujące się w diodach na pasku zczytują informacje w bardzo precyzyjny sposób, więc krytycznie ważne jest odpowiednie generowanie instrukcji.

W tym celu można wykorzystać gotową bibliotekę **Adafruit Neopixel**. Przychodzi ona z wieloma funkcjami, które w prosty sposób umożliwiają sterowanie paskami ledów adresowalnych. Jednak ma ona dwie, dość negatywne właściwości. Przez sposób działania, umożliwia korzystanie z jedynie czterech specyficznych pinów jednocześnie, co oznacza nie więcej niż 4 paski świejące indywidualnie w różny sposób. Oprócz tego, biblioteka ta czerpie z pewnych zasobów pamięci z ograniczeniem dostępu, w rezultacie wymagając dodatkowych pozwoleń przed uruchomieniem programu. To mogłoby sprawiać zgrzyty przy automatycznym starcie serwera.

Aby uniknąć tych problemów, lepszym rozwiązaniem jest napisanie własnej klasy, która będzie zajmowała się generowaniem sygnału. Nie posiada ona wymogów co do typu pinu, dzięki czemu można podłączyć większą liczbę pasków, ani nie wymaga specjalnych pozwoleń, co eliminuje ewentualne problemy z jej wykorzystaniem. Jednak specyfikacja mikrokontrolerów WS2812B stanowi kolejny problem. Mianowicie poprawnie sformułowany sygnał składa się z szeregu wysokich i niskich napięć. Te interwały muszą występować po sobie i każdy trwać określoną długość czasu. Precyzja która jest wymagana sięga aż nanosekund (10^{-9} s). Tak małe wartości nie są możliwe do osiągnięcia przez język Python. W celu zaimplementowania metody generującej sygnały o takich długościach użyto podfunkcji napisanej w języku C. Do opracowania tej metody korzystano z poradnika tłumaczącego zasady działania

sterowników WS2812B [16].

W ten sposób za pomocą klasy `LedStrip.py` można sterować pojedynczym paskiem, tak jak każdym innym Outputem.

4.4.2.3. Pozytywne aspekty rozwiązania

- **Prostota rozbudowy i modyfikacji fizycznego rozłożenia pasków** - Dzięki tej metodzie, każdy pasek jest swoim osobnym Outputem. To sprawia, że aby dodać nowy pasek, wystarczy powiadomić system o nowym Outputcie. Przy modyfikacji, jedynie zmienić dane Outputu.
- **Prawdziwa indywidualność pasków** - Każdy pasek posiada swój własny wątek, przez co planowanie różnych animacji o różnych prędkościach jest bardzo proste.

4.4.2.4. Negatywne aspekty rozwiązania

- **Potrzeba dodatkowego modułu** - Odpowiedni translator jest niezbędny. Niewłaściwe użycie potencjalnie jest w stanie uszkodzić Raspberry Pi poprzez przesłanie 5V na jej pin.
- **Obciążanie serwera** - Animowanie jednego paska zużywa zauważalną ilość zasobów Raspberry Pi. Przy większej ilości system ewidentnie zwalnia, przez co animacje są bardzo powolne. Jeśli użytkownikowi zależy na płynności i prędkości, powinien używać pojedynczego paska lub zrezygnować z tej metody.

4.4.3. Sterownik Arduino

Alternatywnym rozwiązaniem od tego przedstawionej w punkcie 4.4.2 jest użycie osobnego, specjalistycznego kontrolera, którego zadaniem jest jedynie obsługa pasków ledowych. Dzięki temu serwer Raspberry Pi nie musi przejmować się różnicą napięć ani wytwarzaniem wysoce precyzyjnych długości sygnałów. Całą tą odpowiedzialność przerzuca na sterownik, samemu jedynie przekazując komendy od użytkownika gdy zaistnieje taka potrzeba.

Ważną kwestią jest więc wybór odpowiedniego urządzenia sterującego. W projekcie została użyta płytką Arduino Uno, która jest świetnym kontrolerem do pasków WS2812B ze względu na 5V operacyjnych. Przystosowana do Arduino wersja języka C++ jest dużo bardziej optymalna i pozwala na dużo szybsze wykonywanie kodu. Dodatkowo biblioteka `Adafruit Neopixel` na tą właśnie platformę nie posiada podobnych restrykcji jak jej Pythonowy odpowiednik. Lecz pozostają dwie kwestie, które należy poruszyć:

- **Komunikacja z serwerem** - Aby kontroler wiedział jak ma się zachowywać, musi dostawać polecenia od serwera. Omówienie użytej technologii znajduje się w punkcie 4.4.3.1.
- **Program kontrolera** - Urządzenie samo z siebie nie będzie wykonywało żadnych akcji. Kod płytki oraz jego omówienie znajduje się w punkcie 4.4.3.2.

4.4.3.1. Połączenie I2C

Magistrala I2C (Inter-Integrated Circuit) to rodzaj połączenia wykorzystującego tylko dwa kable, a dzięki popularnemu adresowaniu 7-bitowemu można podłączyć i przysyłać dane pomiędzy aż do 128 urządzeń jednocześnie. Architektura sygnału oparta jest o strukturę Master-Slave, co oznacza, iż conajmniej jedno z urządzeń nadaje prędkość przysyłu oraz zarządza połączeniem, natomiast reszta dostosowuje się do jego warunków.

Takie rozwiązanie zakładające że to właśnie serwer jest Masterem nie wymaga dodatkowej translacji napięć, gdyż napięcie będzie dyktowane przez niego. Arduino poradzi sobie z decyfrowaniem przekazu z niższego napięcia.

4.4.3.2. Kod Arduino

Jako że Arduino używa języka C++, nie jest możliwe dynamiczne zmienianie i dodawanie wartości, tak jak to umożliwia Python, albo jest to w znacznym stopniu utrudnione, szczególnie gdy na uwadze trzeba mieć bardzo małą pamięć. Z tego powodu wszystkie paski podpięte do urządzenia są zdefiniowane i wpisane w tablicę.

Praca Arduino to tak naprawdę jedna główna pętla (`loop`), wykonująca się w nieskończoność. To nakłada pewne bariery na sposób, w jaki pisze się kod. W projekcie

wykorzystano podejście tak zwanej "Maszyny Stanów" (rysunek 4.3): zmienna `state` oznacza, w której pozycji z kilku zaprogramowanych znajduje się płytka. To podejście symuluje `mainSummon`.

```
switch(state){
  default:
  case 0: // OFF
    buttonCheck();
    for(int i=0; i<AMM; i++){
      strips[i].clear();
      strips[i].show();
    }
    break;
  case 1: // Default Color
    buttonCheck();
    for(int i=0; i<AMM; i++){
      strips[i].fill(defaultColor,0,LEN);
      strips[i].show();
    }
    break;
  case 2: // Default Swipe
    Swipe(defaultColor,7);
    Swipe(0x000000,7);
    break;
  case 3: // Static Colors
    buttonCheck();
    for(int i=0; i<AMM; i++){
      strips[i].fill(mainColors[i],0,LEN);
      strips[i].show();
    }
    break;
  case 4: // Visor
    Visor(Color(255,0,0),7);
    break;
}
```

Rysunek 4.3. Struktura maszyny stanów.

Jeżeli urządzenie otrzyma komendę od serwera, zostanie ona wpisana w kolejkę komend do wykonania. Kolejka jest konieczna, ponieważ gdyby serwer przesłał więcej poleceń, nie wszystkie mogłyby się wykonać. Instrukcja z początku kolejki zostaje odszyfrowana i wykonana, a następnie program powtarza proces z następną pozycją.

Użytkownik ma możliwość wykonywania animacji na wszystkich paskach w tym samym kolorze, bądź pokolorować każdy pasek indywidualnie. Do tego służą zmienna `defaultColor` oraz tablica `mainColors`.

W projekcie zainstalowano również przycisk, który pozwala użytkownikowi przełączać pomiędzy stanami bez użycia kodu czy aplikacji.

4.4.3.3. Pozytywne aspekty rozwiązania

- **Łatwa instalacja** - Podłączenie wszystkich elementów jest bardzo proste, oraz nie jest potrzebny dodatkowy translator.
- **Obsługa wielu pasków** - Istnieje możliwość podłączenia wielu pasków ledowych jednocześnie.
- **Prędkość wykonywania Instrukcji** - Nawet przy większej ilości pasków, animacje wykonują się bardzo szybko i płynnie.
- **Odciążenie serwera** - Dzięki przekazaniu bardzo ciężkiego na mocy obliczeniowej procesu na inny kontroler, serwer może lepiej pracować.

4.4.3.4. Negatywne aspekty rozwiązania

- **Potrzebny dodatkowy moduł** - Płytką Arduino jest niezbędna i stanowi podstawę tej metody.
- **Trudna rozbudowa** - Zmiana ilości pasków nie jest niemożliwa, ale utrudniona, ponieważ wszystkie animacje są napisane pod konkretną liczbę, co wymaga większej ilości czasu i uwagi przy ewentualnych zmianach.
- **Utrudnione animacje indywidualne** - Jako iż jeden kontroler który nie posiada możliwości śledzenia wielu wątków obsługuje wszystkie paski, projektowanie animacji wykonujących różne czynności na różnych paskach jednocześnie stanowi dość spore wyzwanie.

Niektóre z negatywów można by zlikwidować stosując rozwiązanie, w którym zastosowano by osobny kontroler dla każdego paska. Pozwoliłoby to na prawdziwą wielowątkowość, choć rozbudowa czy aktualizacja oprogramowania byłaby dość żmudna. Niemniej dla użytkownika, który chciałby uzyskać bardziej wydajny system, metoda ta mogłaby się okazać najlepszą.

4.5. Input

4.5.1. Opis klasy

Podobnie jak Output, Input jest klasą która służy jako punkt odniesienia dla serwera oraz fizycznych urządzeń podpiętych do płytki. Jednakże w tym przypadku sygnał idzie w drugą stronę: to urządzenie przesyła dane do serwera, a konkretnie do klasy Calendar uruchamiając Triggery (opisane w punkcie 4.7).

Klasa Input posiada trzy atrybuty:

- **id** - Unikalne ID, służące do identyfikacji i odniesień do konkretnych instancji klasy.
- **name** - Nazwa nadana przez użytkownika w celu ułatwienia mu rozpoznania Inputu.
- **inputClass** - podobnie jak w przypadku Outputu, jest to instancja klasy urządzenia które reprezentuje Input.

Już po samych argumentach można określić że działanie tej klasy jest podobne do klasy Output. Natomiast funkcje są trochę inne:

- **__init__** - Konstruktor, który działa w podobny sposób co konstruktor w klasie Output.
- **startInput** - Ta funkcja uruchamia wątek urządzenia **inputClass**. Założenie jest takie, iż jeżeli dane urządzenie nie jest wykorzystywane (jego informacje nie uruchamiają żadnych zmian) to utrzymywanie jego pracy byłoby marnowaniem zasobów.
- **triggerEvents** - Jest to funkcja którą wywołuje urządzenie **inputClass** kiedy wykryje zmianę. Powiadamia ona o tym klasę Calendar, która wykorzystuje te informacje.

Dzięki tej klasie serwer zbiera dane ze świata rzeczywistego i może na nie odpowiadać. Warto więc omówić zawarte w projekcie klasy urządzeń mogących być użytych jako **inputClass**.

4.5.2. SensorMotion.py

Ta klasa obsługuje czujnik ruchu. W projekcie użyto czujnika **PIR HC-SR501**. Podobnie jak w przypadku **Led.py**, ta klasa ma tylko jeden atrybut w postaci numeru fizycznego pinu który zbiera dane z czujnika. Dopełnieniem klasy są jej funkcje:

- **__init__** - Konstruktor ustawiający podany numer pinu.
- **activate** - Główna funkcja klasy. Uruchamia ona bowiem czujnik, poprzez utworzenie obiektu **MotionSensor** z gotowej biblioteki **gpiozero**, oraz wprawiając w ruch pętlę, która monitoruje zmiany z otoczenia tak długo, jak utworzona flaga **isActive** jest ustawiona jako **True**. Podczas wykrycia zmiany wywołwana jest funkcja **statusChange** z wartością ruchu.
- **statusChange** - Funkcja wywołująca wspomnianą wcześniej funkcję **triggerEvents** u rodzicielskiej klasy **Input**.
- **deactivate** - ustawia flagę **isActive** na **False**, co pozwala zakończyć się pętli **while**, w rezultacie kończąc wątek i wyłączając czujnik.

4.6. Summon

4.6.1. Opis klasy

Mając na przykład gotowy Output klasy **Led.py**, użytkownik chciałby wykonać prostą animację, w której to owa dioda mruga kilkukrotnie, a następnie wykonuje **slideAnimation**. Właśnie do tego służy **Summon**. Jest prostą klasą, która posiada 4 atrybuty:

- **id** - Unikalne ID, służące do identyfikacji oraz odniesień przez serwer.
- **name** - Nazwa nadana przez użytkownika, która ma na celu ułatwienie mu orientacji.
- **device** - ID Outputu przypisanego do danego Summona.

- **actions** - Spis funkcji, które ma wykonać Summon na przypisanym Outputie. Każdy element tablicy to słownik, posiadający klucz **actionName** przechowujący nazwę funkcji, oraz opcjonalny klucz **args**, którym jest tablica argumentów które ta funkcja przyjmuje. Nie jest on wymagany, ponieważ nie każda funkcja będzie przyjmować argumenty.

Widać że jest to struktura podobna do wcześniej opisanych Output i Input. Jest tak, ponieważ te klasy łącznie z Event są czterema głównymi klasami zarządzającymi zmianami do których ma dostęp użytkownik. Z tego powodu muszą być proste i przechowywać tylko niezbędne informacje. Warto jest przyjrzeć się zatem funkcjom:

- **__init__** - Konstruktor, który działa na podobnej zasadzie jak konstruktory Output i Input.
- **mainChecker** - Lambda(rodzaj prostej funkcji), która zwraca **device** instancji, jeżeli zostanie **True** jako argument. W przeciwnym razie zwraca **None**.
- **execute** - Główna funkcja klasy. Po wywołaniu najpierw sprawdza czy Output o podanym ID faktycznie istnieje, a następnie iteruje przez wszystkie elementy tablicy **actions**. Jeżeli klasa Outputu nie może się wywołać, blok **try/catch** wychwytuje to i zwraca odpowiedni komunikat. Istnieje też możliwość wykorzystania funkcji która jest dostępna bez względu na typ klasy, nazwaną **timeout**. Przyjmuje ona jako argument czas w sekundach, a po wywołaniu pauzuje Summon na ten właśnie czas.

4.7. Trigger

4.7.1. Opis klasy

Trigger w przeciwieństwie do poprzednio przedstawionych klas, jest bardziej conceptem. Znaczy to, iż element ten może przyjmować różne postaci. Każdy typ Triggera jest inny, przez co nie ma jednej, głównej klasy przechowującej dane oraz zarządzany podtyp. Każda klasa jest inaczej zbudowana. Poniżej omówione są dwie, zawarte domyślnie w projekcie.

4.7.2. TriggerTime.py

To jest naprawdę nieskomplikowana klasa, która posiada w sobie bardzo prosty, podstawowy konstruktor przypisujący instancji dwie zmienne:

- **time** - Czas dnia, podany w ten sposób: [godziny, minuty, sekundy]. Określa o której godzinie ma się uruchomić sygnał.
- **week** - Siedmioelementowa tablica **True/False**. Odpowiada kolejno siedmiu dniom tygodnia (zaczynając od poniedziałku). Jeżeli wartość dnia wynosi **True**, oznacza to że w tym dniu Trigger ma być aktywny.

4.7.3. TriggerSignal

Również bardzo prosta klasa z podobnym konstruktorem, ale różniąca się swoimi dość ciekawymi atrybutami. Opisuje ona bowiem pobieranie i reagowanie na dane z Inputu, a robi to w następujący sposób:

- **inputId** - ID Inputu, z którego Trigger nasłuchuje zmian.
- **lookingFor** - Określa jakiej wartości oczekuje Trigger. Uruchomi się tylko, gdy właśnie ją przekaże Input.

4.8. Event

4.8.1. Opis klasy

Mając wszystkie elementy opisane we wcześniejszej części rozdziału, można je połączyć w jeden, zawierający w sobie informacje o tym co, jak oraz kiedy zrobić. Event to klasa opisująca wydarzenia, które chce uwydatnić użytkownik. Posiada ona pięć ważnych atrybutów:

- **id** - Unikalne ID, służące do identyfikacji oraz odniesień przez serwer.
- **name** - Nazwa nadana przez użytkownika w celu ułatwienia mu rozpoznania Eventu.

- **isActive** - Wartość stanowiąca o tym, czy dany Event jest obecnie aktywny czy nie.
- **trigger** - Instancja Triggera określająca kiedy ma się wykonać Event.
- **summons** - Lista ID Summonów, które wykonują się po uruchomieniu Eventu.

Zasada działania tej klasy jest bardzo prosta. Aby ją jednak lepiej zrozumieć, warto jest omówić jej funkcje:

- **__init__** - Konstruktor, działający na tych samych zasadach jak konstruktor Outputu, Inputu i Summonu.
- **summonThread** - Jest to funkcja później wywoływana jako wątek. Wykonuje ona po kolei wszystkie Summony dla danego Outputu. Jednocześnie pilnuje, by Calendar wiedział, aby zapauzować **mainSummon** tego właśnie Outputu.
- **start** - Główna funkcja Eventu. Po wywołaniu tworzy **queue** (kolejkę). Jej najważniejszym aspektem jest to, że każdy element tej tablicy to lista Summonów do wykonania na jednym konkretnym Outputcie. To oznacza, że jeśli w **summons** są Summony odwołujące się do czterech różnych Outputów, **queue** będzie miało cztery elementy, każdy Output osobno, i wszystkie Summony ułożone w takiej kolejności, w jakiej były zadeklarowane w **summons**. Jeżeli Summon o podanym ID nie istnieje, zwracany jest odpowiedni komunikat. Po utworzeniu **queue**, dla każdego elementu tworzony jest wątek z funkcji **summonThread**. Event czeka na każdy z tych wątków na zakończenie, aby samemu się zakończyć.

Teraz łatwiej jest zrozumieć zalety Eventu. W jednym miejscu można zarządzać wszystkimi Outputami na raz, oraz kazać im wykonywać pożądane instrukcje po spełnieniu wymagań ustawionych przez Triggery. Event pilnuje przy tym by każdy Summon na tym samym Outputcie wykonywał się jeden po drugim. W przeciwnym razie mogłoby dojść do niechcianych efektów. Widać też prostotę w wykorzystaniu tej klasy.

4.9. Calendar

4.9.1. Opis klasy

Calendar jest jednym z ważniejszych elementów w projekcie. Jest on odpowiedzialny za zarządzanie wszystkimi Eventami, czyli pilnuje by faktycznie się wykonały, oraz utrzymuje porządek w przydzielaniu ID Outputom, Inputom, Summonom i Eventom.

4.9.2. Kolejka Eventów

Każdego dnia o północy (oraz w momencie uruchomienia serwera) tworzona jest lista Eventów, które spełniają dwa warunki: są aktywne oraz klasa ich Triggera to `TriggerTime` z odpowiednim dniem ustawionym na `True`. Następnie lista jest sortowana według czasu ustawionego w Triggerze.

Na końcu uruchamiany jest zegar, który nieustannie sprawdza, czy nastał już czas włączenia pierwszego Eventu w kolejce. Jeżeli tak, to po jego zakończeniu usuwa go z listy. Jeżeli nie, sprawdza czy nastąpił już nowy dzień i należy odświeżyć listę. Jeśli oba przypadki są nieprawdziwe, zegar czeka sekundę i powtarza proces.

Ten mechanizm ma jedną charakterystyczną cechę. Po uruchomieniu serwera, wykonają się wszystkie Eventy które miały się wykonać do tej godziny. Jest to działanie celowe. Ma za zadanie nie pominąć niczego ważnego, co mogłoby się wydarzyć przy ewentualnym restarcie serwera.

4.9.3. Spis Triggerów

Calendar trzyma w sobie informacje o Eventach aktywowanych przez Inputy. Do poprawnego funkcjonowania tego systemu został stworzony słownik `triggers`, który trzyma dane o obserwowanych Inputach i listy Eventów które się aktywują przy konkretnych danych. Ten słownik jest tworzony poprzez funkcję `getMatchingEvents`. Po uruchomieniu serwera i zapełnieniu słownika wszystkie potrzebne Inputy są aktywowane.

4.9.4. Config

Config to bardzo prosta mechanika która upewnia się, że w ID żadnej z głównych klas nie pojawią się błędy. Zapisuje on bowiem najwyższe ID jakie zostało użyte dla każdej z nich. Przy tworzeniu nowych, ten numer zostaje zwiększony i użyty jako ID tej instancji. W ten sposób pilnowane jest, by nigdy nie doszło do sytuacji, że dwa obiekty mają to samo ID, bądź by po usunięciu którejś instancji, nowa zajęła jej miejsce. Takie błędy byłyby krytyczne w prawidłowym funkcjonowaniu serwera.

4.9.5. MainSummons

Jak opisano wcześniej w punkcie 4.3, `mainSummon` to zmienna która pozwala na zaprogramowanie Summonu odtwarzającego się w zapętleniu, gdy żaden inny Summon nie wykonuje się na danym Outputcie w owym czasie. Nie jest on konieczny, dlatego może mieć wartość `None`.

Podczas uruchamiania serwera, tworzona jest lista wszystkich Outputów oraz sprawdzane są ich `mainSummony`. Dla każdego z nich który nie jest `None` uruchamiany jest wątek, który egzekwuje dany Summon w zapętleniu, jeżeli Output nie jest obecnie w użyciu.

4.10. Listy

4.10.1. Opis klasy

Dla każdej z czterech klas, którymi może zarządzać użytkownik (`Event`, `Summon`, `Input`, `Output`) utworzona jest osobna klasa, która kontroluje ich instancje, a także przeprowadza wszelkie żądane zmiany. Są to swojego rodzaju bazy danych. W poprzednich punktach można było zobaczyć odwołania do tych właśnie klas, i to właśnie jest powód. Trzymają one bowiem w sobie spis wszystkich obiektów, i są gotowe wskazać poszukiwany po jego ID.

Jako że wszystkie cztery listy są do siebie bardzo podobne, jako przykład szczegółowo omówiona zostanie tylko klasa `EventsList`.

4.10.2. Importy i atrybuty

Klasa musi importować wszystkie elementy, z których będzie korzystać. W przypadku Eventów, jest to między innymi każdy rodzaj Triggera.

EventsList posiada dwa atrybuty:

- **events** - Lista instancji każdego Eventu stworzonego przez użytkownika.
- **filePath** - Odniesienie do miejsca, w którym zapisany jest plik JSON z danymi wszystkich Eventów.

4.10.3. Zapis i odczyt pliku

Warto zaznaczyć, iż niedopuszczalnym jest wskazać miejsce w projekcie jako wartość stała, gdyż na każdym urządzeniu projekt może się znajdować w innym miejscu oraz w inaczej nazywających się folderach. Dlatego ścieżkę do pliku trzeba wskazać poprzez odniesienie do głównego folderu projektu, a dopiero potem wskazać podfolder oraz nazwę pliku. Poprawny sposób odniesienia przedstawiono poniżej:

```
1 filePath = os.path.dirname(os.path.abspath(sys.argv[0]))+'Schedule/Events.json'
```

Funkcja `saveList` wykorzystuje domyślną metodę języka Python, `open`, która umożliwia pracę z plikiem. Jako argumenty przyjmuje ona ścieżkę do pliku oraz tryb pracy, w tym przypadku `w`, określające zapis. Dzięki tej metodzie można użyć `dump` z biblioteki `json`, która zwraca obiekt Jsonowy, mogący zostać wpisany w plik.

Istotnym jest by w `dump` każdy klucz miał wartość, którą można wpisać w obiekt JSON. Nie wolno zatem przekazywać instancji klas, albowiem będzie to skutkowało błędem. Jako że Event w zmiennej `trigger` używa właśnie obiektu, trzeba go rozbić na język zrozumiały dla JSONa. W tym celu użyte zostały dwa klucze:

- **triggerType** - Jest to nazwa klasy Triggera, otrzymywana za pomocą metody `__class__.__name__` wywołanej z obrabianego obiektu.
- **triggerArgs** - Lista wszystkich argumentów klasy Triggera. Otrzymywana jest poleceniem `__dict__`.

Dzięki takiemu zabiegowi ominięty został problem zapisu do pliku obiektu. Ważne jest jednak przy tym, by pamiętać o dwóch rzeczach: Konstruktor Triggera musi

przyjmować wszystkie uzyskane argumenty, oraz każdy z argumentów musi być wartością możliwą do wpisania w obiekt JSON.

Na bardzo podobnych zasadach działa funkcja `loadList`, która podając `r` metodzie `open` odczytuje przy pomocy `json.load` a następnie tworzy listę Eventów. Warto zauważyć, iż dla każdego Eventu najpierw tworzony jest obiekt `Trigger` z obu omówionych wcześniej kluczy. Później jest on podawany w konstruktorze Eventu wraz z innymi potrzebnymi im danymi. Na końcu Event zostaje dodany do listy.

Przed załadowaniem Eventów należy się upewnić, iż `Summony` oraz `Inputy` są już załadowane, by uniknąć problemów wynikających z ich braku w liście.

W przypadku gdy użytkownik chciałby wyczyścić całą listę, metoda `deleteList` wpisuje pustą listę JSON w plik.

4.10.4. Znajdowanie obiektów z listy

Lista ma na celu ułatwić innym klasom szybki dostęp do konkretnych instancji tylko po ich ID czy nazwie. Dlatego powstały następujące metody:

- **getEventbyName** - Iteruje przez listę Eventów i zwraca pierwszy, który ma podaną nazwę.
- **getEventbyId** - Iteruje przez listę Eventów i zwraca pierwszy, który ma podane ID. Ta metoda jest skuteczniejsza, gdyż dzięki `Configowi` nie dojdzie do sytuacji, w której zostanie zwrócony zły obiekt, co mogłoby się przytrafić, jeśli dwa Eventy mają tę samą nazwę.

4.10.5. Zarządzanie obiektami

Ważną funkcją list jest możliwość dodawania, modyfikowania oraz usuwania elementów z listy. Do tego powstały następujące funkcje:

- **addEvent** - Funkcja tworząca nowy Event, działająca na podobnych zasadach co `loadList`. Jej dodatkową funkcją jest jednak sprawdzanie czy podane `Summony` faktycznie istnieją. Jeżeli nie, zwraca błąd.
- **deleteEvent** - Funkcja całkowicie usuwająca Event o danym ID.

- **modifyEvent** - Funkcja która zmienia poszczególne atrybuty danego Eventu. Warto przy tym zaznaczyć, iż nie jest możliwym zmienić ID Eventu, co oczywiście jest celowe.

Każda z opisanych funkcji po wykonaniu zapisuje stan listy do pliku, oraz wywołuje metodę `sendOk`, która jest konieczna w komunikacji z aplikacją, która została omówiona w punkcie 4.12.

4.11. ConnectionHandler

4.11.1. Opis klasy

ConnectionHandler to statyczna klasa mająca zadanie zarządzać internetową częścią serwera, czyli połączeniem z klientem (aplikacją Android) przez system TCP Socket.

Socket - *punkt końcowy dwukierunkowego kanału komunikacyjnego pomiędzy serwerem a klientem. Sockety mogą komunikować się w ramach procesu, pomiędzy procesami na tej samej maszynie lub pomiędzy procesami na różnych maszynach. Aby móc komunikować się ze zdalnym programem, musimy połączyć się poprzez port Socketu.* [17]

4.11.2. Obsługa połączenia klienta

Po uruchomieniu Socketu serwera poleceniem `StartServer` nasłuchuje on nadchodzących połączeń od klientów. Dozwolone jest tylko jedno połączenie, gdyż tak został przystosowany serwer, i większa ilość klientów niechybnie by spowodowała błędy. Po udanym nawiązaniu połączenia, każda nadchodząca informacja jest przekazywana dalszej części programu, zajmującej się jej deszyfrowaniem 4.11.3. Ten etap może chcieć, a często wręcz musi, przesyłać klientowi informacje zwrotne. Wykorzystuje się do tego jedną z trzech komend:

- **sendOk** - Wysyła potwierdzenie poprawnego wykonania polecenia, na przykład dodania nowej pozycji w bazie danych.
- **sendMessage** - Wysyła spersonalizowaną wiadomość do klienta.
- **sendJsonData** - Wysyła dane w postaci obiektu JSON.

Każda informacja przesłana z użyciem tych komend musi mieć na końcu znak nowej linii, ponieważ to jest właśnie sygnał którego wyczekuje funkcja czytająca dane z Socketu klienta w aplikacji mobilnej. Dlatego każda z funkcji upewnia się by ten znak tam się znajdował.

4.11.3. Komendy

Użytkownik chcąc wprowadzić pewne zmiany bądź tylko wyświetlić zawartość bazy danych (list) musi swe intencje przekazać serwerowi. Aby ten wiedział, co ma robić, wprowadzono system komend, które mieszczą się w słowniku `commandDictionary`. Jako iż język Python nie posiada funkcji `SWITCH`, zazwyczaj zastępuje się ją blokiem poleceń `if/else`. Jednak nie jest to rozwiązanie optymalne, szczególnie gdy założenia projekt ma mieć możliwość znacznej rozbudowy, która z pewnością zwiększyłaby ilość komend. Dlatego słownik okazał się dobrym kandydatem. Pozwala on wskazać potrzebną komendę otrzymaną przez `RequestDecoder`. Następnie uruchamiana jest funkcja zlokalizowana jako wartość pod poszukiwanym kluczem.

Jednak pewnien problem pojawia się w momencie, gdy to użytkownik przesyła dane, a nie tylko nazwę komendy. Dla przykładu, gdyby chciano dodać nowy Event, najpierw wezwana byłaby odpowiednia komenda, a dopiero kolejny przypływ danych zawierałby w sobie wartości nowego Eventu. Aby zaradzić temu w jak najprostszy sposób, komendy, które wymagają dodatkowych danych są w `commandDictionary` wpisane jako lambda. Dzięki temu są w stanie oczekiwać na potrzebne informacje już w samym momencie wywołania, co znacznie upraszcza nie tylko konstrukcję klasy, jak i dodawanie kolejnych komend.

4.12. Aplikacja Android

Tworzenie aplikacji mobilnych to dość skomplikowany proces, wymagający bardzo konkretnych klas i metod. Z tego powodu warto omówić najważniejsze aspekty oraz sposoby jej projektowania przed przedstawieniem poszczególnych klas i mechanizmów.

- **Aktywność** - Jest to główny element z którego składa się aplikacja. Każda aktywność opisuje co w danym momencie dzieje się na ekranie. Zawiera ona przeróżne elementy, takie jak przyciski, napisy, obrazki czy omawiane niżej fragmenty. Aktywności są swego rodzaju pojedynczymi programami, przez co wymiana danych pomiędzy wieloma z nich jest skomplikowana, dlatego przyjmuje się zasadę iż każda aktywność powinna skupiać się na jednym aspekcie aplikacji, za to być jak najbardziej rozbudowana.
- **Layout** - Każda aktywność, fragment oraz inne elementy prezentujące się w jakiś graficzny sposób w aplikacji muszą mieć swój własny layout. Jest to plik w formacie xml, opisujący w jaki sposób dany moduł wygląda, co się na nim znajduje i jaką ma funkcjonalność. Aby skorzystać z przycisku w aktywności, jej layout musi zawierać o nim informacje, takie jak gdzie się znajduje, jak wygląda i jaką funkcję wywołuje jego wciśnięcie.
- **Fragment** - Jest to swego rodzaju aktywność zagnieżdżona w innej aktywności. Dzięki takiemu podejściu możliwe jest tworzenie bardziej złożonych aplikacji, reagujących na informacje w bardzo dynamiczny sposób. Jako że wczytywanie aktywności może być dość długie, umieszczenie jej jako fragment w innej pozwoli użytkownikowi na korzystanie z reszty funkcji na ekranie w czasie gdy komputer załadowuje potrzebny fragment.

4.12.1. MainActivity: Ekran główny

Główny ekran stanowią przyciski otwierające poszczególne sekcje, czyli ustawienia, listy Eventów, Summonów, Outputów i Inputów oraz menu zarządzania nimi. Rysunek 4.4a przedstawia rozmieszczenie przycisków na ekranie.

4.12.2. SettingsActivity: Ustawienia

Ustawienia to miejsce w którym użytkownik może połączyć się z serwerem, podając jego adres IP oraz port. Po nawiązaniu połączenia możliwe jest zsynchronizowanie danych, czyli pobranie z serwera list wszystkich Eventów, Summonów, Outputów i Inputów. Bez tego listy będą puste. Możliwe jest też wysłanie komendy na uruchomienie danego Eventu czy Summonu, wykonania funkcji na Outpucie bądź zasymulowanie sygnału z Inputu. Użytkownik dowiaduje się o pomyślności przeprowadzonej akcji poprzez odpowiednie komunikaty. Na rysunku 4.4b przedstawiono wygląd menu ustawień.

4.12.3. Listy pozycji

Po udanej synchronizacji i powrocie na ekran główny kliknięcie w przycisk odpowiadający jednej z klas wyświetli listę wszystkich instancji wraz z ich danymi. Rysunek 4.4c przedstawia sposób wyświetlania informacji o Eventach. Aktywność każdej z pozostałych trzech klas wygląda bardzo podobnie, różniąc się jedynie danymi każdej pozycji, gdyż oczywiście każda klasa ma inne atrybuty.

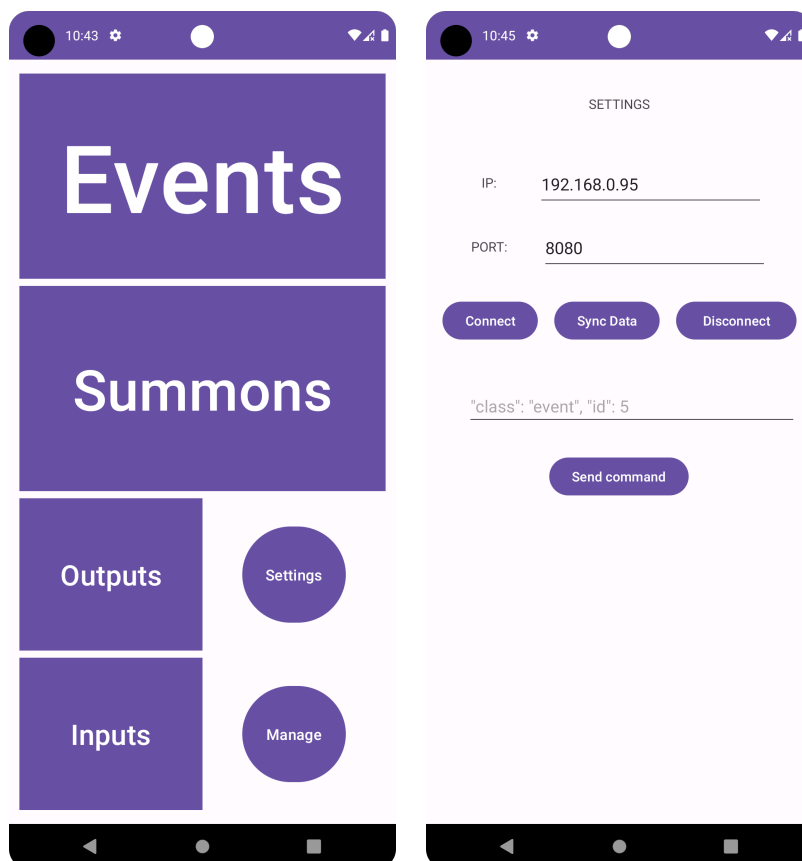
Jako iż użytkownik ma możliwość dodawania nowych pozycji w nieskończoność, pewna jest sytuacja, w której to nie zmieszczą się one na ekranie jednocześnie. Dlatego do list zaimplementowano mechanizm RecyclerView. Pozwala on na wyświetlanie elementów z listy, ale o wiele bardziej optymalnie niż w przypadku zwykłego wyświetlania. Gdyby lista zawierała na przykład 1000 pozycji, zwykłe podejście załadowałoby je wszystkie do pamięci, tak by można je było wyświetlić przesuwając listę w dół. To ogromnie by obciążało telefon oraz mogłoby doprowadzić do błędów i zawieszenia się aplikacji. RecyclerView rozwiązuje ten problem, tworząc tylko tyle instancji, by zapełnić cały ekran. Gdy potrzebne są nowe pozycje, przesuwa element który zniknął z ekranu, zmienia jego dane na potrzebne do wyświetlenia i ponownie pokazuje tą instancję, prezentującą nowe wartości. Podczas implementacji tego mechanizmu wspomagano się poradnikiem [18], który świetnie opisuje sposób działania RecyclerView.

4.12.4. AddNewActivity: Menu zarządzania

AddNewActivity to menu w którym można dodawać, modyfikować bądź usuwać pozycje z list. Działa ono na bazie fragmentów, w której to wybór klasy z pierwszej listy aktywuje pożądany fragment. Każdy z nich zawiera opcje atrybutów danej klasy, które są aktywne w zależności od wybranej czynności, co przedstawia rysunek 4.4d. Widać na nim wybrany Output jako klasę którą użytkownik chce zarządzać. Z kolei ustawienie operacji na dodanie nowej pozycji wyłącza możliwość edycji ID. To pokazuje iż ta pozycja nie jest brana pod uwagę (ponieważ przy tworzeniu nowego Outputu ID jest przypisywane automatycznie).

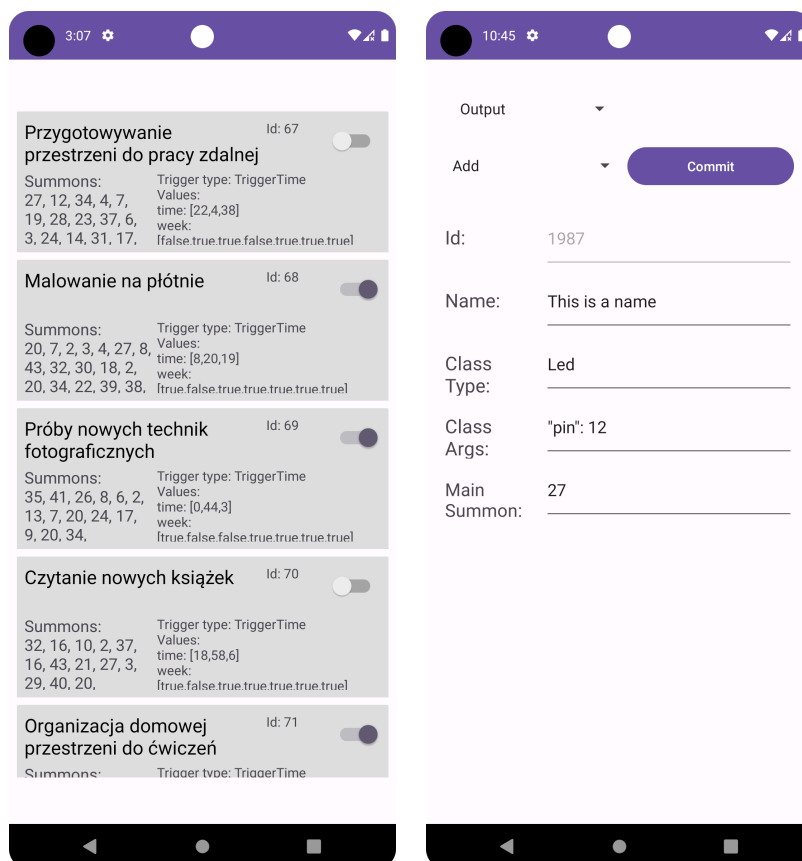
4.12.5. SocketManager

SocketManager to statyczna klasa działająca w tle podczas użytkowania aplikacji. Odpowiada za zarządzanie połączeniem z serwerem, pozwala reszcie klas porozumiewać się z nim i prosić o informacje oraz przetrzymuje dane list. Statyczna klasa nie wymaga inicjowania, a jej publiczne funkcje i atrybuty są dostępne dla każdej innej klasy z tego samego pakietu. Jest to przydatne, gdy dla przykładu pewna funkcja potrzebuje danych z serwera. Może je uzyskać korzystając z funkcji `sendMessage`, bądź specjalnie przygotowanej struktury zadania, na przykład `SyncTask`, pobierającego dane o klasach. Ważnym jest, iż aby jakaś komenda się wykonała, musi być ona zaprogramowana w `ConnectionHandlerze` serwera 4.11.3.



(a) MainActivity

(b) SettingsActivity



(c) EventsActivity

(d) AddNewActivity

Rysunek 4.4. Wygląd ekranów aplikacji.

Rozdział 5

TODO Dyskusja rezultatów i wnioski końcowe

5.1. Porównanie rezultatów z oczekiwaniami

Niniejszy materiał miał na celu przedstawienie koncepcji Smart Home oraz omówienie związanych z nią kwestii, takich jak co może ona oferować, jakie są podejścia jej realizacji oraz dla jakich osób jest każda z nich. Na bazie tych informacji zaprezentowano projekt stanowiący przykład niestandardowego rozwiązania. Głównymi aspektami, nad którymi się skupiał były czytelność i przejrzystość dla użytkownika o każdym stopniu doświadczenia z programowaniem oraz elektryką oraz zbudowanie solidnej bazy pozwalającej na dobrowolną i łatwą rozbudowę. Będąc w stanie dowolnie przeglądać zrozumiały kod oprogramowania, użytkownik miałby pełne zaufanie co do bezpieczeństwa jego danych, co rzadko ma miejsce przy standardowych podejściach. Aby udowodnić praktyczność tej koncepcji, należało zaprojektować system posiadający pewne funkcjonalności.

Pierwszym punktem stanowiącym o prawidłowym wykonaniu zadania jest posiadanie odpowiednich elementów: podświetlenia adresowalnych pasków LED, czujników ruchu oraz silników krokowych. Wszystkie z wymienionych modułów zostały zawarte w przykładowej instalacji oraz dokładnie opisane w rozdziale 4. Zaprogramowano wsparcie do tych urządzeń oraz przedstawiono sposób ich użytkowania. Każde z wyżej wymienionych narzędzi uzyskało własną klasę, napisaną w sposób modu-

łowy, oparty na strukturze pozwalającej skutecznie rozbudowę i uprościć obsługę zarówno tych, jak i nowych elementów.

Następnym wymaganiem była kontrola nad kolorami oraz jasnością poszczególnych pasków ledowych oraz zapewnienie użytkownikowi możliwości dodawania własnych animacji oraz sekwencji animacji. Zostało to poruszone w punkcie 4.4, prezentującym różne podejścia do wykorzystywania takich pasków. Użytkownik może zmieniać kolory oraz jasność każdego z nich, a dzięki Summonom opisanym w punkcie 4.6 jest w stanie planować całe sekwencje własnoręcznie zaprogramowanych animacji.

Użytkownik miał też mieć możliwość kontroli nad roletami okiennymi za pomocą silników krokowych. Punkt 4.3.3 traktuje właśnie o tym urządzeniu. Omawiane są też w nim różne sposoby na osiągnięcie tego właśnie celu.

Ostatnim z zadań które należało wykonać była aplikacja na telefon z systemem Android. Miała ona umożliwiać użytkownikowi sterowanie układem i zarządzać wydarzeniami wykonującymi się o odpowiednim czasie bądź reagujących na dane z czujników. Obie formy aktywacji zostały zaprogramowane jako Triggery omówione w punkcie 4.7, i służą one do aktywacji Eventów opisanych w punkcie 4.8. Aplikacja zawierająca aktywności odpowiadające za poszczególne zadania związane z zarządzaniem Eventami, Summonami, Outputami i Inputami jest dokładnie przedstawiona w punkcie 4.12.

5.2. Wnioski

Jest wiele możliwych rozwiązań dla osób zainteresowanych własną instalacją Smart Home, między którymi jest ten projekt. Spełnił on swoje założenia, mimo iż niektóre z jego elementów stanowią jedynie minimalną funkcjonalność. Posiada on dużo możliwości rozbudowy, na przykład w zakresie aplikacji, która po kilku usprawnieniach mogłoby dużo zyskać na przyjemności z korzystania. Sposób, w jaki używane są w układzie paski led zostawia dużo do życzenia. Jest on w stanie zapewnić podstawowe funkcje, lecz obsługa i rozbudowa jest zbyt toporna by można ją uznać za gotową. Jednak pewne silne aspekty projektu świadczą o jego potencjale. Solidna podstawa złożona z Outputów, Inputów, Summonów i Eventów, przecho-

wywanie danych oraz Calendar spełniają swoje zadanie.

Reasumując wszystkie kwestie kwintesencji tematu, dochodzę do fundamentalnej konkluzji. Projekt osiągnął sukces sprostając postawionymi przed sobą zadaniami. Jest on unikalny, z założeniami skierowanymi ku użytkownikowi. Udało się utworzyć system Smart Home, który może być wykorzystywany i rozbudowywany.

Bibliografia

- [1] Researchgate.net. https://www.researchgate.net/figure/Double-drum-winder-configuration_fig1_260404317.
- [2] Bankmycell.com. How many smartphones are in the world?
<https://www.bankmycell.com/blog/how-many-phones-are-in-the-world>.
- [3] Adam Hayes. Smart home: Definition, how they work, pros and cons.
Investopedia.com, 9 2023.
<https://www.investopedia.com/terms/s/smart-home.asp>.
- [4] Statista Research Department. Number of users of smart homes worldwide from 2019 to 2028. *Statista.com*, 11 2023.
<https://www.statista.com/forecasts/887613/number-of-smart-homes-in-the-smart-home-market-in-the-world>.
- [5] Parker Hall. The best smart speakers with alexa, google assistant, and siri.
Wired.com, 10 2023.
<https://www.wired.com/story/best-smart-speakers/>.
- [6] Mindy Woodall. The most popular smart home devices. *Reviews.org*, 6 2023.
<https://www.reviews.org/home-security/most-popular-smart-home-device-statistics/>.
- [7] Jason Cohen. Amazon's alexa collects more of your data than any other smart assistant. *Pcmag.com*, 3 2022. <https://www.pcmag.com/news/amazons-alexa-collects-more-of-your-data-than-any-other-smart-assistant>.

- [8] Victoria Giardina. The 20 best smart home devices on amazon in 2023, per reviews. *Nypost.com*, 2 2023.
<https://nypost.com/article/best-smart-home-devices-on-amazon/>.
- [9] Harbingers.io. Open source.
<https://harbingers.io/definicje/open-source>.
- [10] Wikipedia. Wejście-wyjście ogólnego przeznaczenia. https://pl.wikipedia.org/wiki/Wejście-wyjście_ogólnego_przeznaczenia.
- [11] Vibha Gupta. 10 best programming languages for raspberry pi. *Almabetter.com*, 8 2023. <https://www.almabetter.com/bytes/articles/best-programming-languages-for-raspberry-pi>.
- [12] Sruthi Veeraraghavan. Top 20 best programming languages to learn in 2024. *Simplilearn.com*, 12 2023. <https://www.simplilearn.com/best-programming-languages-start-learning-today-article>.
- [13] Mahesh Chand. Most popular databases in the world (2023). *C-sharpcorner.com*, 8 2023. <https://www.c-sharpcorner.com/article/what-is-the-most-popular-database-in-the-world/>.
- [14] Wikipedia. Zintegrowane środowisko programistyczne. https://pl.wikipedia.org/wiki/Zintegrowane_Środowisko_programistyczne.
- [15] Gaven MacDonald. *Stepper Motor Control with the Raspberry Pi*. <https://youtu.be/Dc16mKFA7Fo>, 7 2013.
- [16] Gil Vargas. *User request: Neopixel/WS2812 timing explained*. <https://youtu.be/q00cv8BImz8>, 12 2022.
- [17] DigitalOcean Pankaj. *Python Socket Programming - Server, Client Example*. <https://www.digitalocean.com/community/tutorials/python-socket-programming-server-client>, 8 2022.
- [18] Practical Coding. *RecyclerView / Everything You Need to Know*. <https://youtu.be/Mc0XT58A1Z4>, 10 2021.